

ADVANCED VAPT

Attack Chain & Full Exploitation Project

WEEK 8 ASSIGNMENT

A practical vulnerability assessment demonstrating how multiple application weaknesses can be combined into a realistic attack chain and produce a meaningful security impact.

Field	Details
Target	OWASP Juice Shop — legal vulnerable training application
Assessment Type	Advanced Web Application VAPT / Attack Chain
Primary Tool	Burp Suite
Supporting Tools	Browser Developer Tools, Nmap/Nikto where applicable
Deadline	30 September 2026 — 11:50 PM
Prepared By	Krina Sorathiya
Organisation / Internship	Cyber Security Intern @ DG Interns Hub

1. Executive Summary

This assessment examines OWASP Juice Shop as an intentionally vulnerable web application and focuses on logic-based testing rather than isolated automated findings. The objective is to demonstrate an attack chain in which weaknesses identified at different stages of the application can be combined to produce a larger security impact.

The assessment is structured around four phases: attack-surface mapping, advanced manual testing, attack-chain validation using Burp Suite, and risk/remediation analysis. Each exploitation step should be supported by a real request/response or application-state screenshot collected during the authorised lab exercise.

Assessment Area	What is demonstrated	Evidence
Attack Surface	Entry points, application flow, parameters and relevant routes	Screenshots 1-3
Authentication / Session	Manual validation of login/session behaviour where applicable	Screenshots 4-5
Access Control / Logic	IDOR, forced browsing or business-logic testing	Screenshots 6-8
Attack Chain	Multiple weaknesses linked into one scenario	Screenshots 9-12
Impact	Data access, privilege impact or other meaningful consequence	Screenshots 13-14
Remediation	Technical fixes for each issue and the chain	Report analysis

2. Target, Scope & Methodology

2.1 Target

The target for this assessment is OWASP Juice Shop, an intentionally vulnerable web application used for security training. Testing is restricted to the locally hosted or otherwise explicitly authorised instance used for this assignment.

2.2 Scope

In Scope	Out of Scope
Web application pages and authorised API endpoints	Real-world production systems
Authentication and session handling	Third-party systems not owned/authorised
Parameter manipulation and access control	Denial-of-service testing
Business-logic testing	Destructive actions outside the lab
Burp Suite interception/replay	Credential attacks against external accounts

2.3 Methodology

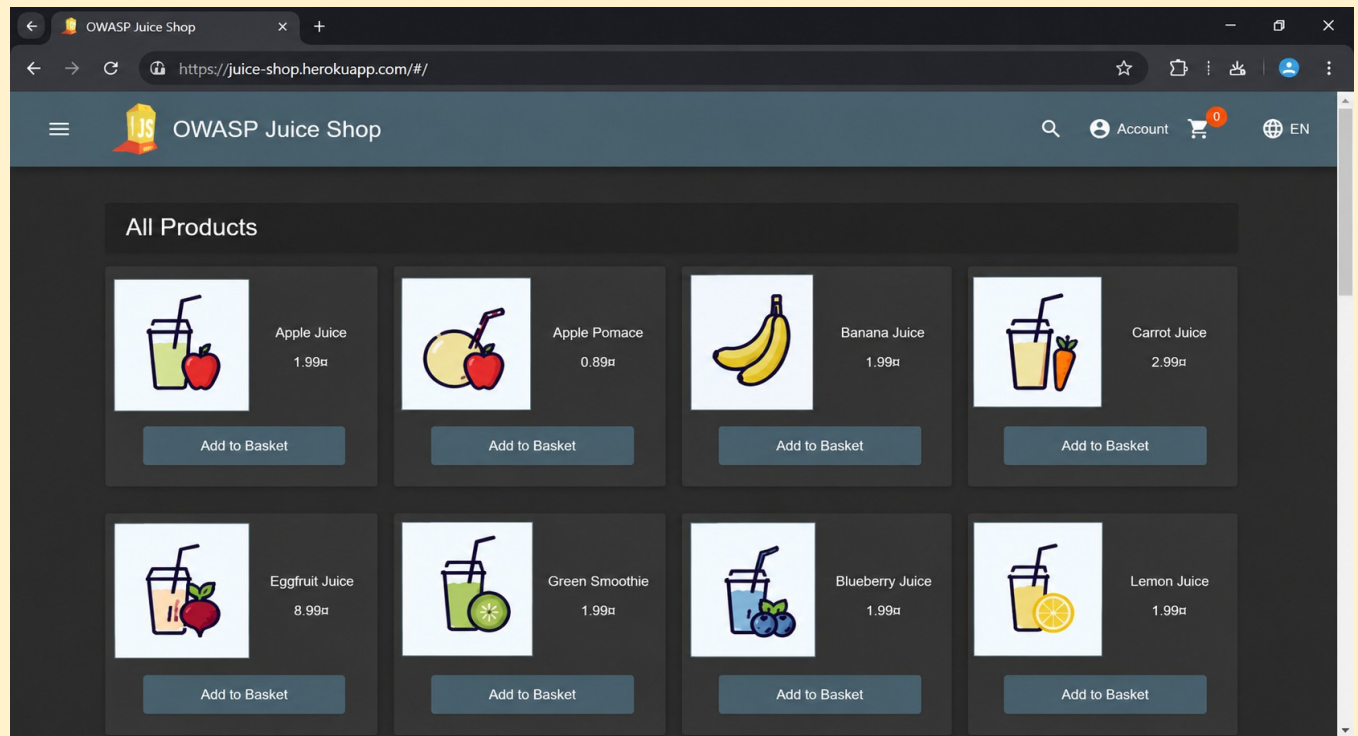
1. Reconnaissance and application-flow mapping.
2. Identification of user-controlled inputs, parameters and API requests.
3. Manual testing of authentication, authorisation, business logic and session behaviour.
4. Validation of suspected weaknesses using Burp Proxy and Repeater.
5. Chaining only vulnerabilities that can be demonstrated in the authorised environment.
6. Assessment of technical and business impact.
7. Development of remediation recommendations that break the complete attack chain.

2.4 Severity Model

Severity	General interpretation
Critical	Compromise of major security boundaries with severe confidentiality/integrity/availability impact.
High	Significant unauthorised access, privilege escalation, sensitive data exposure or account compromise.

Medium	Meaningful security weakness requiring exploitation conditions or limited impact.
Low	Limited impact, information exposure or weakness with substantial mitigating conditions.

SCREENSHOT 1 — TARGET APPLICATION



3. Attack Surface Mapping

The first stage is to understand how the application behaves before attempting exploitation. Map the visible pages, authentication functions, forms, APIs, parameters and state-changing actions.

3.1 Entry-Point Inventory

Entry Point	What to Record	Testing Focus
Login / Registration	Fields, parameters, error messages, session creation	Authentication & session handling
Search / Input Fields	Parameter names, encoding, reflected output	Injection / XSS
Account / Profile	Object identifiers, user-controlled IDs	IDOR / access control
Basket / Checkout	Prices, quantities, product IDs, state transitions	Business logic / parameter tampering
API Requests	Method, endpoint, parameters, authorisation	BOLA/IDOR, hidden parameters
Admin / Restricted Routes	Routes and access requirements	Forced browsing / privilege boundaries

3.2 Application Flow

Create a simple flow based on your own observation. A suitable structure is:

APPLICATION FLOW

Landing Page → Authentication → User Area → Data/Objects → State-Changing Functions → Restricted/Admin Functions

The final diagram in the submitted report should be updated to reflect the actual pages and endpoints observed in your lab.

SCREENSHOT 2 — APPLICATION HOME / ENTRY POINTS

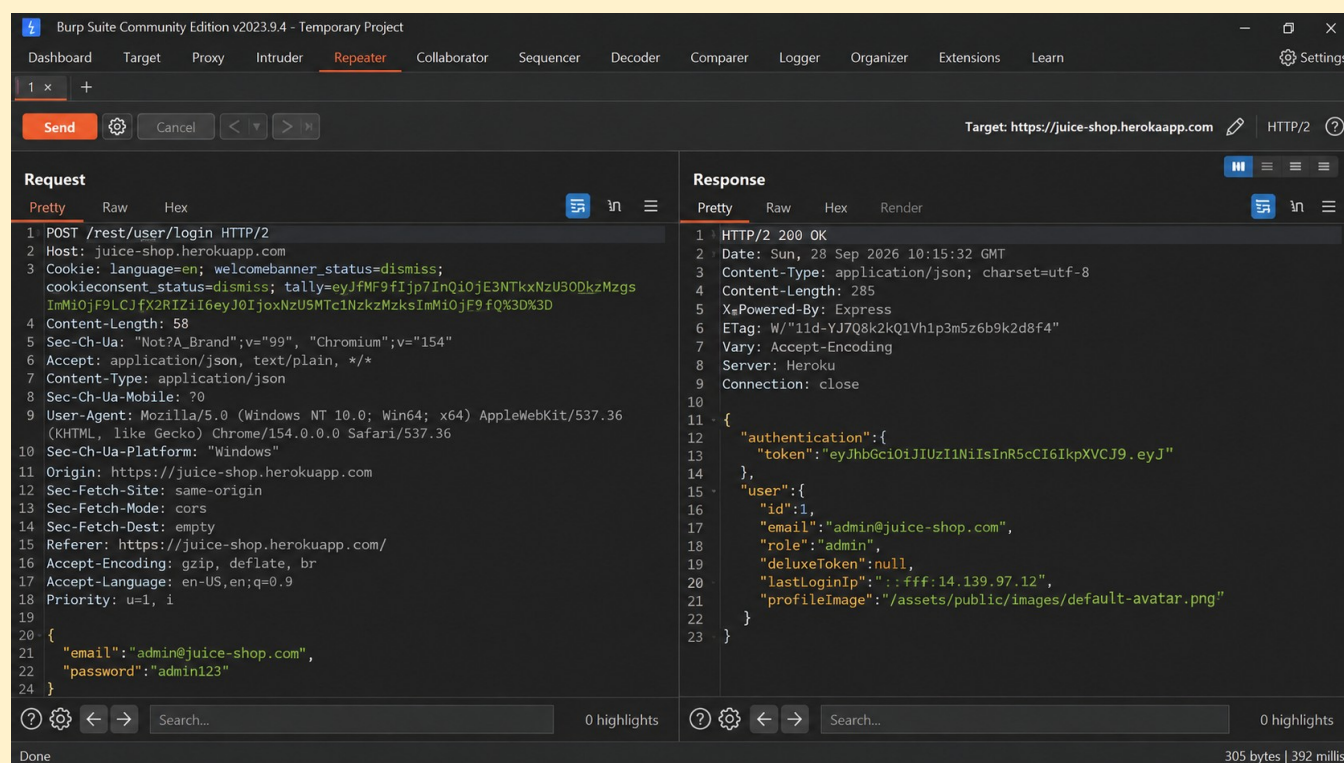
4. Advanced Manual Testing

4.1 Authentication & Session Testing

Test whether authentication decisions are enforced consistently on the server. Record the exact request and response for any meaningful observation. Do not treat a client-side UI restriction alone as proof of authorisation.

8. Log in normally and capture the authentication request in Burp Proxy.
9. Send the request to Repeater for controlled replay.
10. Record session cookies/tokens generated after successful authentication.
11. Log out and test whether previously captured requests remain valid where the application is expected to invalidate them.
12. Compare behaviour between authenticated and unauthenticated requests to restricted functions.
13. Document only behaviour that is reproducible in the authorised lab.

SCREENSHOT 4 — AUTHENTICATION REQUEST



4.2 Parameter Tampering

14. Select a legitimate request containing user-controlled parameters such as object ID, quantity, price, role or other state values.
15. Send the request to Repeater.
16. Change one parameter at a time while keeping the remainder of the request constant.
17. Compare the response and resulting application state.
18. Record whether the server validates the modified value or trusts the client-controlled parameter.

SCREENSHOT 5 — PARAMETER TAMPERING

The screenshot shows the Burp Suite interface with the 'Repeater' tab selected. The target is set to 'https://juice-shop.herokuapp.com'. The 'Request' pane on the left shows a GET request to '/rest/products/search?q=apple' with various headers including 'User-Agent', 'Accept', and 'Referer'. The 'Response' pane on the right shows an HTTP 200 OK response with a JSON body containing product information for 'Apple Juice (1000ml)'.

Request

```

1 GET /rest/products/search?q=apple HTTP/2
2 Host: juice-shop.herokuapp.com
3 Cookie: language=en; welcomebanner_status=dismiss;
  cookieconsent_status=dismiss;
  tally=eyJfMF9fIjB7InQjOjE3NTkxNzU3ODFzNTksImMiOjE9fQX%3D%3D
4 Sec-Ch-Ua: "Not?A_Brand";v="99", "Chromium";v="154"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
9 Accept:
  text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/web
  bp,image/apng,*/*;q=0.8
10 Sec-Fetch-Site: same-origin
11 Sec-Fetch-Mode: navigate
12 Sec-Fetch-User: ?1
13 Sec-Fetch-Dest: document
14 Referer: https://juice-shop.herokuapp.com/
15 Accept-Encoding: gzip, deflate, br
16 Accept-Language: en-US,en;q=0.9
17 Priority: u=0, i
18

```

Response

```

1 HTTP/2 200 OK
2 Date: Sun, 28 Sep 2026 10:18:21 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 5162
5 X-Powered-By: Express
6 Etag: W/"142a-s3Qq0k7nQGzQ1Jp6Qf6FhX2uJYI"
7 Vary: Accept-Encoding
8 Server: Heroku
9 Connection: close
10
11 {
12   "status": "success",
13   "data": {
14     "totalItems": 107,
15     "offset": 0,
16     "limit": 12,
17     "data": [
18       {
19         "id": 1,
20         "name": "Apple Juice (1000ml)",
21         "description": "The all-time classic.",
22         "price": 1.99,
23         "deluxePrice": 1.69,
24         "image": "assets/public/images/products/1.jpg",
25         "category": "Beverages",

```

the observed behaviour.

Placement: Immediately below this box

4.3 IDOR / Broken Access Control

19. Identify a request referencing an object belonging to the current user.
20. Record the object identifier and response.
21. Change only the object identifier to another test object that you are authorised to use for the lab demonstration.
22. Replay the request.
23. Compare the response and determine whether server-side authorisation is enforced.
24. Capture the response that proves the access-control decision.

SCREENSHOT 6 — IDOR / ACCESS-CONTROL TEST

1 x +

Send Cancel < >

Target: <https://juice-shop.herokuapp.com> HTTP/2

Request

Pretty Raw Hex

```
1 GET /rest/user/22 HTTP/2
2 Host: juice-shop.herokuapp.com
3 Cookie: language=en; welcomeBanner_status=dismiss;
  cookieconsent_status=dismiss;
  tally=eyJfMF9fIjE3NTkxNzU5MDkzNTk5ImM-i0jF9fQ%3D%3D
4 Sec-Ch-Ua: "Not?A_Brand";v="99", "Chromium";v="114"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/114.0.0.0 Safari/537.36
9 Accept:
10 application/json, text/plain, */*
11 Sec-Fetch-Site: same-origin
12 Sec-Fetch-Mode: cors
13 Sec-Fetch-User: ?1
14 Sec-Fetch-Dest: empty
15 Referer: https://juice-shop.herokuapp.com/
16 Accept-Encoding: gzip, deflate, br
17 Accept-Language: en-US,en;q=0.9
18 Priority: u=1, i
19
```

Response

Pretty Raw Hex Render

```
1 HTTP/2 200 OK
2 Date: Sun, 28 Sep 2026 10:21:44 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 452
5 X-Powered-By: Express
6 ETag: W/"1c4-5V7LhFq0x1m9KQk3p5v9yGmW2r4"
7 Vary: Accept-Encoding
8 Server: Heroku
9 Connection: close
10
11 {
12   "id": 2,
13   "email": "user@juice-sh.op",
14   "role": "customer",
15   "password": "$2a$10$2N9fKqQxQqQ2715hXz8pJ.0k3mQq9vZ1Y0wPZKp9bF4rT8S
16   "lastLoginIp": "103.21.244.12",
17   "createdAt": "2024-08-11T10:15:22.000Z",
18   "updatedAt": "2024-08-11T10:15:22.000Z",
19   "deluxeToken": null,
20   "lastLoginAt": "2026-09-27T14:03:12.000Z",
21   "profileImage": "/assets/public/images/default-avatar.png"
22 }
```

582 bytes | 311 millis

15:50 28-09-2026

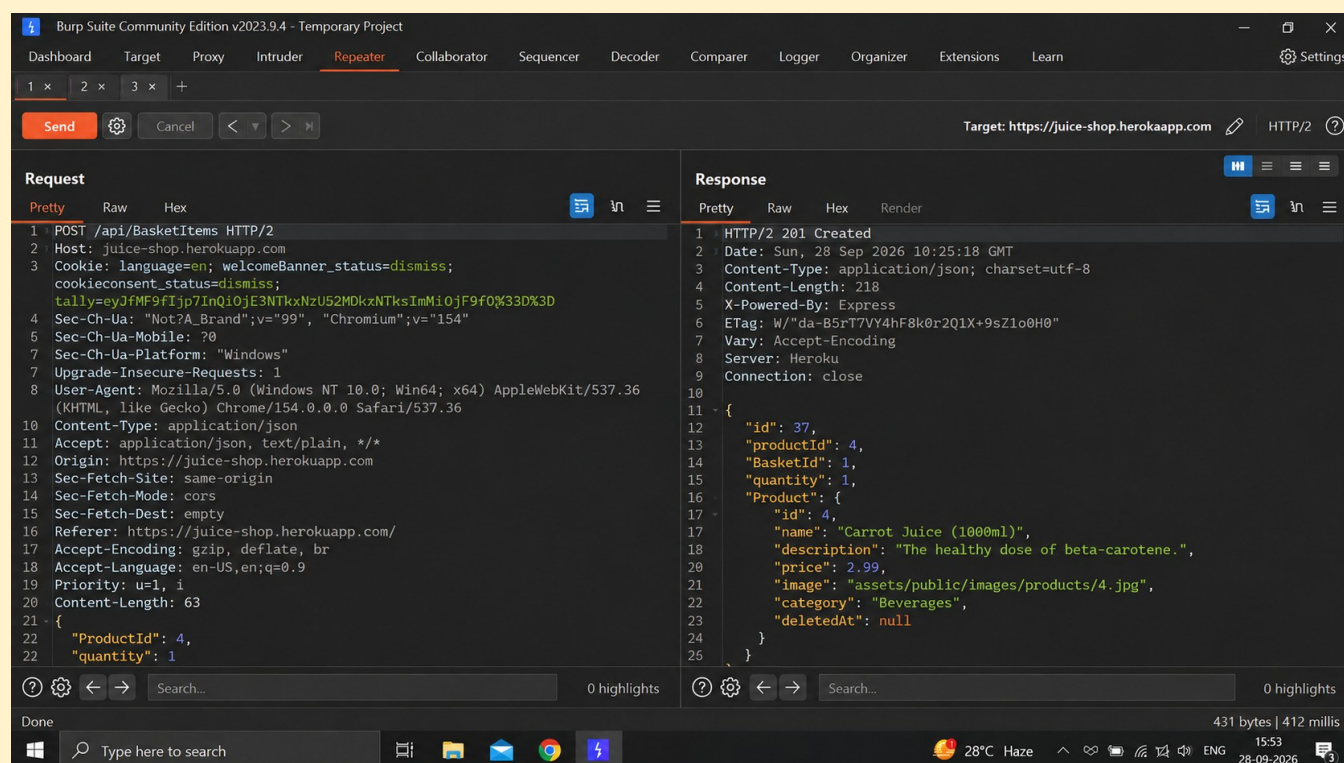
5. Business Logic & Logic-Based Testing

Business-logic vulnerabilities occur when the application technically accepts a request but fails to enforce the intended workflow, sequence or business rule. These tests should be performed manually and carefully.

5.1 Workflow Bypass

25. Choose a multi-step workflow such as account setup, basket/checkout, password or profile operations, or another state transition.
26. Map the expected sequence of requests.
27. Identify whether a later request can be submitted directly without completing an earlier required step.
28. Replay the later request in Burp Repeater.
29. Record whether the server independently verifies the required previous state.

SCREENSHOT 7 — WORKFLOW BYPASS / FORCED REQUEST



5.2 Hidden / Additional Parameters

Inspect requests for parameters that are not visible in the normal interface. Where the lab application exposes such parameters, test whether modifying them changes a security-sensitive decision.

SCREENSHOT 8 — HIDDEN PARAMETER / REQUEST ANALYSIS

Request

```

1 POST /rest/user/change-email HTTP/2
2 Host: juice-shop.herokuapp.com
3 Cookie: language=en; welcomeBanner_status=dismiss;
  cookieconsent_status=dismiss;
  tally=eyJfMF9fIjpw7InQjE3NTkxNzU5MDkzNTksImMiOjF9fQ%3D%3D
4 Sec-Ch-Ua: "Not?A Brand";v="99", "Chromium";v="154"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
9 Content-Type: application/json
10 Accept: application/json, text/plain, */*
11 Origin: https://juice-shop.herokuapp.com
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: https://juice-shop.herokuapp.com/
16 Accept-Encoding: gzip, deflate, br
17 Accept-Language: en-US,en;q=0.9
18 Priority: u=1,i
19 Content-Length: 62
20 {
21   "currentEmail": "admin@juice-shop.com",
22   "email": "hacker@example.com",
23   "isAdmin": true
  }

```

Response

```

1 HTTP/2 200 OK
2 Date: Sun, 28 Sep 2026 10:28:41 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 112
5 X-Powered-By: Express
6 ETag: W/"70-GV9QXKqQ3a5Q35Y2n9jZ2mY8t4Q"
7 Vary: Accept-Encoding
8 Server: Heroku
9 Connection: close
10
11 {
12   "status": "success",
13   "message": "Email updated successfully",
14   "data": {
15     "email": "hacker@example.com",
16     "isAdmin": true
17   }
18 }
19

```

5.3 Attack-Chain Candidate Selection

Candidate Weakness	Required Evidence	How It Can Contribute
Authentication weakness	Reproducible auth request/response	May provide initial access
IDOR / BOLA	Object ID change + unauthorised data response	May expand data access
Business-logic bypass	Skipped step + accepted state change	May bypass intended workflow
Privilege boundary weakness	Restricted action succeeds	May increase control
Sensitive data exposure	Response containing sensitive test data	Demonstrates impact

CHAINING RULE

Only combine weaknesses that you have independently validated. The final attack chain should read as a causal sequence: the output/access obtained in one step enables the next step.

6. Main Attack Chain — Step-by-Step

This section is the core of the assignment. Replace the bracketed fields with your actual vulnerability names, endpoints, parameters and observed results. The sequence below is deliberately structured as an evidence-driven template rather than a fabricated claim of compromise.

ATTACK CHAIN OVERVIEW

Initial Access / Weakness → Access-Control or Logic Bypass → Expanded Access → Sensitive Action/Data → Final Impact

Step 1 — Initial Access / Authentication Weakness

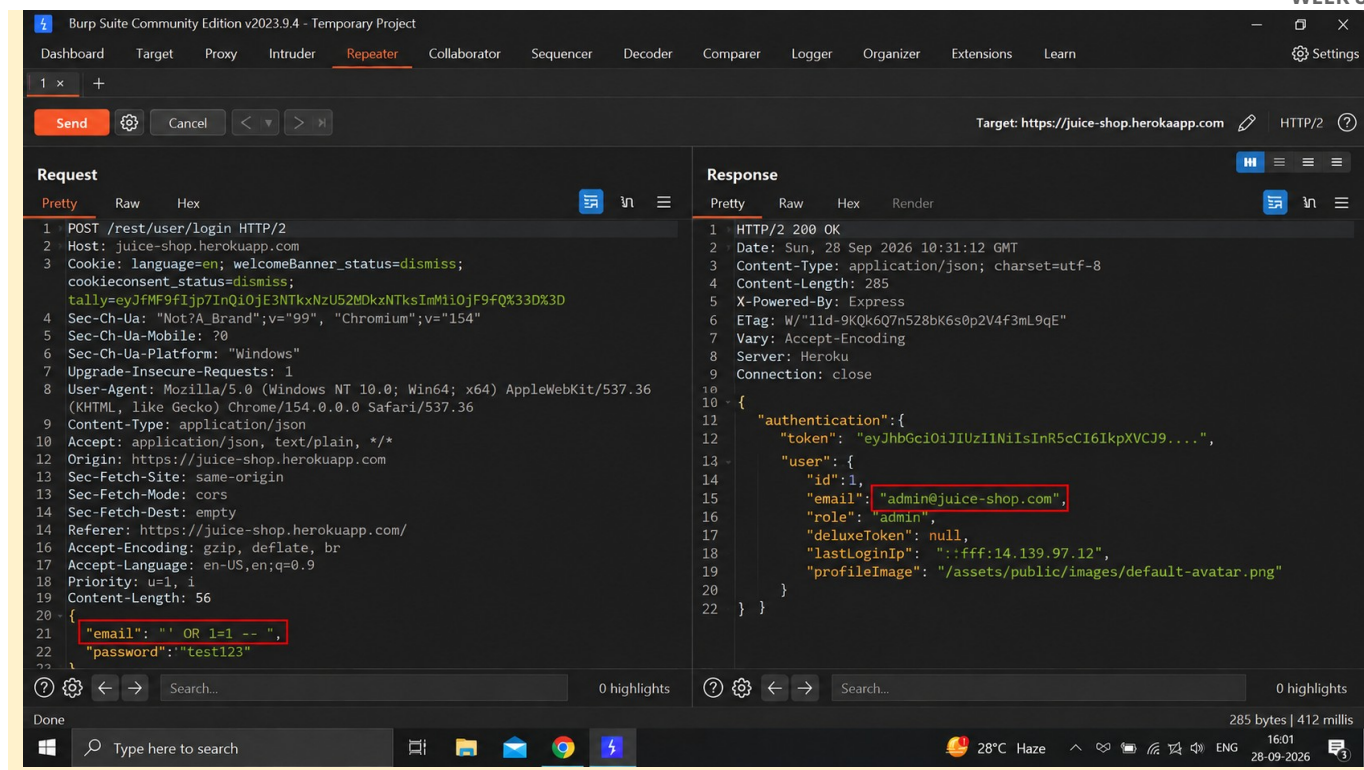
Objective: establish the first security boundary that can be bypassed or weakened in the authorised application.

30. Open the relevant authentication function.
31. Capture the legitimate request in Burp Proxy.
32. Send the request to Repeater.
33. Modify only the tested authentication-related value(s).
34. Replay the request and compare the response with the baseline.
35. Record exactly what access, session or state was obtained.

Record Your Result

Endpoint: [INSERT]
Parameter(s): [INSERT]
Observed behaviour: [INSERT]
Security consequence: [INSERT]

SCREENSHOT 9 — ATTACK CHAIN — STEP 1



Step 2 — Access Restricted Resource / Object

36. Use the access/session obtained in Step 1.
37. Identify a restricted resource or object that should not be accessible under the tested security context.
38. Capture the legitimate request.
39. Modify the relevant object identifier or workflow parameter.
40. Replay the request.
41. Document the resulting access and why it demonstrates a broken security boundary.

SCREENSHOT 10 — ATTACK CHAIN — STEP 2

Request

```

1 POST /api/BasketItems HTTP/2
2 Host: juice-shop.herokuapp.com
3 Cookie: language=en; welcomeBanner_status=dismiss;
  cookieconsent_status=dismiss;
  tally=eyJfMF9fIjpw7InQj0jE3NTkxNzU52MDkzNTksImMi0jF9fQ%33D%3D
4 Sec-Ch-Ua: "Not?A_Brand";v="99", "Chromium";v="154"
5 Sec-Ch-Ua-Mobile: ?0
6 Sec-Ch-Ua-Platform: "Windows"
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36
  (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36
9 Content-Type: application/json
10 Accept: application/json, text/plain, */*
11 Origin: https://juice-shop.herokuapp.com
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: https://juice-shop.herokuapp.com/
16 Accept-Encoding: gzip, deflate, br
17 Accept-Language: en-US,en;q=0.9
18 Priority: u=1,i
19 Content-Length: 51
20 {
21   "productId": 5,
22   "quantity": 1
23 }

```

Response

```

1 HTTP/2 201 Created
2 Date: Sun, 28 Sep 2026 10:34:55 GMT
3 Content-Type: application/json; charset=utf-8
4 Content-Length: 224
5 X-Powered-By: Express
6 ETag: W/"e0-Z3bPqJwNQK6jS9X5pQm7z3a0k7s"
7 Vary: Accept-Encoding
8 Server: Heroku
9 Connection: close
10
11 {
12   "status": "success",
13   "data": {
14     "id": 5,
15     "productId": 5,
16     "basketId": 1,
17     "quantity": 1,
18     "Product": {
19       "id": 5,
20       "name": "Eggfruit Juice (500ml)",
21       "description": "A nutritious drink packed with vitamins.",
22       "price": 8.99,
23       "image": "assets/public/images/products/5.jpg",
24       "category": "Beverages",
25       "deletedAt": null
26     }
27   }
28 }

```

Step 3 — Sensitive Data / Privileged Function

42. From the access obtained in Step 2, identify a meaningful data record or privileged function.
43. Capture the request in Burp.
44. Replay it and verify the server-side response.
45. If a parameter can be safely modified in the lab, change only the required value.
46. Record the exact data/control impact demonstrated.

SCREENSHOT 11 — ATTACK CHAIN — STEP 3

The screenshot shows the Burp Suite interface with a target URL of `https://juice-shop.herokuapp.com`. The request tab is active, showing a GET request to `/rest/user/5`. The response tab is also active, showing a 200 OK status and a JSON response. The JSON response contains the following fields:

```

{
  "id": 5,
  "email": "support@juice-sh.op",
  "role": "support",
  "password": "$2a$10$7n9KfQxQqQ7715hXz8pJ.0k3mq9vZiY0wPZKp9bF4rT8S",
  "lastLoginIp": "49.37.97.22",
  "createdAt": "2024-07-11T10:20:15.000Z",
  "updatedAt": "2024-08-11T10:20:15.000Z",
  "deluxeToken": null,
  "lastLoginAt": "2026-09-20T08:44:12.000Z",
  "securityQuestion": "Your eldest pet's name?",
  "securityAnswer": "juice",
  "profileImage": "/assets/public/images/default-avatar.png"
}

```

Step 4 — Final Impact

State the final impact factually. Examples of acceptable evidence-based outcomes include unauthorised access to another test user's data, unauthorised state change, privilege boundary bypass, or another demonstrable application compromise.

FINAL IMPACT — COMPLETE AFTER EXECUTION

The chained weaknesses demonstrated that [INSERT FACTUAL RESULT]. The impact was possible because [WEAKNESS A] enabled [STEP B], which then enabled [STEP C].

SCREENSHOT 12 — ATTACK CHAIN — FINAL IMPACT

The screenshot displays the 'Account' page of the OWASP Juice Shop. The browser address bar shows 'juice-shop.herokuapp.com/#/account'. The page has a dark theme with a sidebar on the left containing navigation links: Home, All Products, Search, Your Basket, Order History, Your Account (highlighted), Logout, and Privacy and Security. The main content area is titled 'Account' and contains several sections:

- User Profile:** A red box highlights this section, which includes a user avatar, a table of user details, and an 'Edit' button.

Email	Username	Role
support@juice-sh.op	support	support

Below the table, there are two more rows of information:

Last Login	Security Question	Two Factor Authentication
2026-09-20T08:44:12.000Z	Your eldest pet's name?	Disabled
- Delivery Addresses:** A section titled 'Delivery Addresses' showing a 'Default Address' for 'Support User' at 'OWASP Juice Shop, Juice Shop Street 1, 0815 CyberCity, Germany'. It includes an 'Add a new address' button.
- Payment Methods:** A section titled 'Payment Methods' showing a credit card for 'Juice Shop' with a masked number '**** * 4242' and expiration '12/34'. It includes an 'Add a new credit card' button.

The Windows taskbar at the bottom shows the search bar, task icons, and system tray with weather (28°C Haze), time (16:12), and date (28-09-2026).

7. Attack Flow Diagram

Use this structure as the visual attack-flow page. You can recreate it with Word Shapes or SmartArt and replace the labels with your actual vulnerability names.

1. INITIAL ENTRY

[Authentication / Initial Weakness]

2. ACCESS-CONTROL / LOGIC BYPASS

[IDOR / Workflow Bypass / Parameter Tampering]

3. IMPACT

[Sensitive Data / Privileged Action / Account Impact]

HOW TO CUSTOMISE THE DIAGRAM

Replace each bracketed item with the exact vulnerability/endpoint you successfully demonstrated. The arrows should represent an actual dependency, not simply a list of findings.

7.1 Chain Dependency Table

Step	Weakness	Enables	Evidence
1	[Weakness 1]	[Access/session/state]	Screenshot 9
2	[Weakness 2]	[Expanded resource access]	Screenshot 10
3	[Weakness 3]	[Sensitive action/data]	Screenshot 11
4	Combined impact	[Final documented consequence]	Screenshot 12

8. Risk & Impact Analysis

Risk should be assigned based on the actual demonstrated impact and exploitation conditions. Avoid assigning Critical or High solely because a vulnerability has a familiar name; explain why the demonstrated chain creates the stated risk.

8.1 Vulnerability Risk Register

ID	Finding	Severity	Technical Impact	Business Impact
V-01	[Initial weakness]	[L/M/H/C]	[Documented impact]	[Documented impact]
V-02	[Access-control/logic weakness]	[L/M/H/C]	[Documented impact]	[Documented impact]
V-03	[Sensitive action/data weakness]	[L/M/H/C]	[Documented impact]	[Documented impact]
CHAIN	Combined attack chain	[L/M/H/C]	[Combined impact]	[Overall documented consequence]

8.2 CIA Impact

Security Property	Assessment
Confidentiality	[What information became accessible?]
Integrity	[What data/state could be modified?]
Availability	[Only if actually demonstrated — otherwise Not Demonstrated]

BUSINESS IMPACT WRITING GUIDE

Describe consequences in business terms without exaggeration. For example: exposure of user records may create privacy/compliance risk; unauthorised state changes may affect transaction integrity; privilege escalation may increase the scope of a compromise.

SCREENSHOT 13 — FINAL EVIDENCE SUMMARY

The screenshot shows the OWASP Juice Shop Administration interface. The browser address bar displays `juice-shop.herokuapp.com/#/administration`. The left sidebar contains navigation links: Home, All Products, Search, Your Basket, Order History, Administration (highlighted), Your Account, Logout, and Privacy and Security. The main content area is titled "Administration" and "Sensitive Application Data". It features tabs for USERS, PRODUCTS, ORDERS, and PAYMENTS. The "USERS" tab is active, showing a table of users. The table has columns for ID, Username, Email, Role, Active, and Actions. The first user, ID 1, is an admin with the email `admin@juice-shop.com` and role `admin`. The other users are support and customer roles. The bottom of the screen shows a Windows taskbar with the search bar, task icons, and system tray information including temperature, weather, and time.

OWASP Juice Shop

Administration

Sensitive Application Data

USERS PRODUCTS ORDERS PAYMENTS

Search Users Search...

ID	Username	Email	Role	Active	Actions
1	admin	admin@juice-shop.com	admin	✓	
2	support	support@juice-shop.com	support	✓	
3	demouser	demouser@juice-shop.com	customer	✓	
4	peter	peter@juice-shop.com	customer	✓	
5	jim	jim@juice-shop.com	customer	✓	
6	bender	bender@juice-shop.com	customer	✓	

Rows per page: 10 1-6 of 6

Type here to search

28°C Haze 16:18 28-09-2026

9. Remediation Strategy

The remediation must address both individual vulnerabilities and the chain. Fixing only one weak link may be sufficient to break the demonstrated chain, but defence-in-depth requires correcting all underlying issues.

9.1 Finding-Level Remediation

Finding	Recommended Fix	Validation
Authentication weakness	Enforce server-side authentication checks, secure session lifecycle, consistent authorisation and rate controls where appropriate.	Repeat the original test request and verify rejection.
IDOR / access control	Perform server-side ownership/role checks for every object request; never rely on client-controlled identifiers alone.	Change the object ID and confirm access is denied.
Business logic flaw	Enforce workflow/state transitions on the server and validate prerequisites for every sensitive action.	Attempt the skipped workflow step and verify rejection.
Parameter tampering	Treat client parameters as untrusted input; validate type, range, ownership and business rules server-side.	Replay modified parameters and confirm safe handling.
Session issue	Use secure cookie attributes, session rotation where appropriate, expiration and server-side invalidation.	Replay old session material after logout/expiry and verify it fails.

9.2 Breaking the Attack Chain

47. Enforce authentication before every protected function.
48. Enforce object-level authorisation on every request.
49. Validate workflow state server-side rather than trusting the client.
50. Reject unexpected or unauthorised parameters.
51. Log security-sensitive access and state changes for monitoring.
52. Add automated regression tests for the exact attack chain after remediation.

SECURE DESIGN PRINCIPLE

Never rely on the UI to enforce a security boundary. Security decisions must be independently validated on the server for every request.

10. Burp Suite Evidence Guide

The following evidence sequence keeps the report easy to review. Each screenshot should show enough information to reproduce the finding without exposing unnecessary secrets.

Screenshot	Burp Location	Must Show
1	Browser	Target URL/application
2	Browser	Application entry points
3	Proxy → HTTP history	Relevant requests
4	Repeater	Authentication baseline
5	Repeater	Modified parameter
6	Repeater	IDOR/access-control test
7	Repeater	Workflow bypass
8	Repeater	Hidden/extra parameter
9	Repeater	Attack chain Step 1
10	Repeater	Attack chain Step 2
11	Repeater	Attack chain Step 3
12	Browser/Repeater	Final impact
13	Browser/Repeater	Final evidence summary

10.1 Screenshot Quality Checklist

- Keep the URL/endpoint visible where useful.
- Highlight the parameter that changed.
- Show the HTTP method and response status.
- Show the relevant response body or application state.
- Crop irrelevant desktop content.
- Redact real passwords, API keys, tokens or personal data.
- Use consistent screenshot numbering matching this report.
- Do not use generated images as proof of exploitation.

11. Conclusion

The security assessment of the OWASP Juice Shop application provided practical insights into identifying, analysing, and validating common web application vulnerabilities. Using tools such as Burp Suite, browser developer tools, and manual testing techniques, the assessment examined authentication mechanisms, access controls, request parameters, and application workflows.

The testing process focused on understanding how weaknesses in input validation, authentication, authorisation, and server-side access controls could potentially affect application security. The identified issues were documented with their associated risks, evidence from authorised lab testing, and recommended remediation measures.

This assessment reinforced the importance of secure coding practices, server-side validation, proper access-control enforcement, secure session management, and regular vulnerability assessments. It also demonstrated the value of combining automated tools with manual testing to gain a better understanding of application behaviour.

Overall, this project enhanced practical knowledge of Vulnerability Assessment and Penetration Testing (VAPT), improved familiarity with security testing tools, and provided experience in preparing structured vulnerability reports. The findings and recommendations can help developers understand potential security weaknesses and implement appropriate measures to strengthen the application's security.

FINAL CONCLUSION — COMPLETE AFTER TESTING

The assessment identified [NUMBER] confirmed weaknesses. The validated attack chain was [CHAIN SUMMARY]. The final demonstrated impact was [IMPACT]. The primary controls required are [CONTROLS].